HAM-GR: Hierarchical Attentive Memory with Gated Revision for Long-Form Conversational Agents

Kaibo Zhang [1] **Colby Smith** [1] **Annie Liu** [1] **Nelson Lu** [1]

Abstract

Long-form conversational AI systems must maintain accurate and updatable knowledge about entities discussed across extended dialogue histories. Current flat retrieval-augmented memory systems struggle with belief revision: when a fact changes mid-conversation, stale values remain retrievable, confusable entity names introduce cross-contamination, and the system cannot reliably distinguish unsupported facts from updated ones. We introduce HAM-GR, Hierarchical Attentive Memory with conflict-aware Gated Revision, a structured external memory architecture that separates retrieval routing from belief revision. HAM-GR organizes claims into a non-parametric entity-attribute-claim hierarchy and applies a learned soft gate at insertion time to revise leaf embeddings when new claims overwrite existing memory states. A conflict-aware gate further modulates forgetting under semantically contradictory updates, allowing the memory to suppress outdated information more aggressively when revision is required. On a synthetic benchmark of 7,000 multi-turn episodes averaging 109 turns, HAM-GR achieves 81% validation accuracy, compared with 63% for a strong non-parametric overwrite baseline and 51% for flat retrieval, while reducing stale retrieval rate by 91% compared to strict overwrite.

1. Introduction

Long-horizon conversational agents require memory that persists beyond a single context window. Prior work has therefore explored external memory, retrieval, reflection, and context management as mechanisms for extending LLM agents across long documents, multi-session dialogue, and interactive environments (Lewis et al., 2020; Park et al., 2023; Packer et al., 2023; Lee et al., 2024). However, increasing the amount of accessible context does not by itself solve conversational memory: models can fail to reliably use relevant information in long inputs (Liu et al., 2024), and benchmarks for long-term assistants show that knowledge updates and abstention remain core unsolved abilities (Wu et al., 2025; Hu et al., 2026).

We focus on a specific failure mode: belief revision. In a long conversation, a user may first state that Alice lives in Boston and later state that Alice moved to Seattle. A flat retrieval memory can store both claims, but semantic similarity alone does not indicate which claim is current. The same issue appears when similar entities are discussed, when unrelated facts should be preserved, or when a query asks about information that was never stated. Thus, reliable conversational memory must support four coupled behaviors: update tracking, fact preservation, entity disambiguation, and abstention.

Existing memory systems provide important ingredients but do not directly solve this revision problem. Retrieval-augmented generation connects generation to non-parametric memory (Lewis et al., 2020); MemGPT manages information across memory tiers to overcome finite context windows (Packer et al., 2023); Generative Agents and MemoryBank store and retrieve long-term experiences for planning and personalization (Park et al., 2023; Zhong et al., 2024); and recent agentic memory systems organize memory into evolving structures (Xu et al., 2025). These approaches motivate structured long-term memory, but most memory updates are append-based, summary-based, or heuristic. They do not provide a trainable write-time operator that decides how an existing memory state should change when a new contradictory claim arrives.

We propose HAM-GR, Hierarchical Attentive Memory with conflict-aware Gated Revision, a structured external memory architecture for mutable conversational knowledge. HAM-GR has three components: (1) a non-parametric entity-attribute-claim hierarchy that routes claims through structured memory rather than a flat vector index; (2) a learned soft-gated revision module that updates leaf embeddings at insertion time when a new claim overwrites an existing memory state; and (3) a learned attention module that ranks retrieved candidates using the revised embeddings, allowing retrieval supervision to shape the revision gate.

The key idea is that memory should encode not only claim

content, but also the revision history of the memory state. When a new claim is inserted into an existing leaf, HAM-GR produces a revised embedding $\mathbf{h}_{\text{new}}$ by blending the previous state and the incoming claim through learned forget and input gates. A conflict-aware variant further modulates forgetting according to semantic contradiction, so outdated state can be suppressed more aggressively when an update directly conflicts with an older claim.

We evaluate HAM-GR on a synthetic benchmark of 7,000 multi-turn conversational episodes averaging 109 turns. HAM-GR achieves 81% validation accuracy, compared with 63% for a strong non-parametric overwrite baseline and 51% for flat retrieval, while reducing stale retrieval rate by 91%. These results suggest that long-term conversational memory requires not only better retrieval, but also structured and trainable revision.

2. Related Work

2.1. Retrieval and Long-Context Memory

Retrieval-augmented generation connects language generation to an explicit non-parametric memory, typically implemented as a dense index over external documents (Lewis et al., 2020). This formulation has become a standard way to provide LLMs with information beyond their parametric weights. However, standard RAG is primarily a read-time mechanism: it retrieves semantically relevant passages from a static or append-only store, but does not specify how stored beliefs should be revised when later evidence supersedes earlier evidence. This limitation is especially important in long conversations, where the memory may contain multiple claims about the same entity and attribute.

Long-context models partially reduce the need for retrieval, but they do not remove the memory problem. Prior work shows that LLMs can fail to robustly use information in long inputs, especially when relevant evidence appears in difficult positions within the context (Liu et al., 2024). ReadAgent addresses this issue by constructing gist memories and revisiting detailed passages when needed, improving reasoning over very long contexts (Lee et al., 2024). These approaches motivate compact memory representations, but they focus mainly on compressing or accessing long context. HAM-GR instead focuses on mutable memory: when a new claim changes a previously stored fact, the memory state itself should be revised rather than merely appended or summarized.

2.2. Memory Management for Multi-Session Agents

Several agent systems use external memory to support long-horizon interaction. MemGPT frames LLM memory as an operating-system-like context management problem, where information is moved between limited working context and external storage (Packer et al., 2023). This provides an important abstraction for multi-session interaction, but the central operation is memory management across tiers rather than differentiable revision of stored entries. HAM-GR keeps the external-memory framing, but introduces a trainable write-time operator that updates existing memory states when new claims arrive.

Generative Agents store an experience memory stream in natural language, retrieve relevant memories for planning, and periodically synthesize higher-level reflections (Park et al., 2023). Reflexion similarly stores verbal feedback in an episodic memory buffer to improve later decisions without updating model weights (Shinn et al., 2023). MemoryBank extends long-term user memory through storage, recall, update, and user-profile adaptation (Zhong et al., 2024). These systems show that memory is useful for planning, personalization, and interactive behavior. However, their updates are largely append-based, reflection-based, or heuristic. HAM-GR differs by treating revision as a learned operation over memory embeddings, allowing retrieval supervision to shape how old and new claims are combined.

2.3. Structured and Evolving Agent Memory

Recent work argues that memory should be organized rather than stored as a flat list. A-MEM proposes agentic memory inspired by Zettelkasten, dynamically constructing notes, linking related memories, and evolving existing memory representations as new memories arrive (Xu et al., 2025). HippoRAG 2 similarly moves beyond flat vector retrieval by combining dense retrieval with structured propagation to improve factual, associative, and sense-making memory (Jiménez Gutiérrez et al., 2025). These systems support our motivation that memory structure matters: retrieval over a flat vector store can miss important relationships or retrieve semantically related but incorrect evidence.

HAM-GR is complementary to this line of work. Rather than improving only read-time traversal or agent-driven linking, HAM-GR introduces a structured write-time mechanism. Claims are routed through a non-parametric entity-attribute-claim hierarchy, and when a new claim reaches an existing leaf, a learned gate produces a revised embedding. This makes memory evolution differentiable and local: the model learns how a specific memory state should change under updates, instead of relying on global rewriting, heuristic linking, or purely graph-based propagation.

2.4. Reflection, Granularity, and Adaptive Retrieval

Dialogue memory systems must also choose the right granularity of storage and retrieval. Reflective Memory Management addresses this by using prospective reflection to summarize interactions across utterance, turn, and session levels, and retrospective reflection to refine retrieval using evidence

feedback (Tan et al., 2025). This line of work shows that fixed memory granularity and fixed retrieval rules are insufficient for personalized dialogue. However, reflection-based systems still primarily revise memory through summarization and reranking.

HAM-GR addresses a different part of the memory pipeline. Instead of periodically summarizing conversation history or refining retrieval after generation, it revises the memory representation at insertion time. The gated revision module decides how much of the previous memory state to retain and how much of the incoming claim to incorporate. The conflict-aware variant further adjusts forgetting when old and new claims are semantically contradictory. This gives the model an explicit mechanism for reducing stale recall while preserving unrelated facts.

2.5. Benchmarks for Long-Term Memory and Revision

Recent benchmarks make clear that long-term memory is not only about retrieving relevant facts. LongMemEval evaluates long-term assistant memory across information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention (Wu et al., 2025). MemoryAgentBench evaluates memory agents through incremental multi-turn interactions and emphasizes competencies such as accurate retrieval, test-time learning, long-range understanding, and selective forgetting (Hu et al., 2026). These benchmarks reveal that current systems still struggle with updates, unsupported queries, and long histories.

Our synthetic benchmark follows this direction but isolates belief revision in a controlled setting. We evaluate four behaviors: update tracking, fact preservation, entity disambiguation, and abstention. In addition to final accuracy, we measure stale retrieval rate and contradiction rate, which directly quantify whether the memory retrieves outdated or conflicting evidence. This evaluation is designed to test the mechanism HAM-GR targets: not only whether the system can find relevant memories, but whether its stored memory states remain current as the conversation changes.

3. Method

3.1. Problem Formulation

We study conversational memory under mutable facts. An episode consists of a sequence of turns $\mathcal{T} = (t_1, \ldots, t_T)$. A subset of turns contains structured claims $c_t = (e_t, a_t, v_t)$, where e_t is an entity, a_t is an attribute, and v_t is the value currently asserted for that entity-attribute pair. The system maintains an external memory state $\mathcal{M}_t$ after each turn and answers a set of queries $\mathcal{Q}$ after the episode.

For a query $q \in \mathcal{Q}$, we assume the target entity and attribute (e_q, a_q) are identifiable from the query text. The correct answer is the most recent value asserted for this pair:

$$\hat{v}(q) = v_{t^*}, \qquad t^* = \max\{t : (e_t, a_t) = (e_q, a_q)\}.$$

If no claim about (e_q, a_q) has appeared, then $\hat{v}(q) = \varnothing$ and the system should abstain.

The central operation is belief revision. A new claim $c_t = (e, a, v_t)$ supersedes an earlier claim $c_s = (e, a, v_s)$ when both share the same entity-attribute pair and $s < t$. In this case, the memory should reflect v_t as the current value and suppress v_s during future retrieval. We use the term belief revision in the broad sense of updating a belief state after new information arrives (Alchourrón et al., 1985), but instantiate it here as an embedding-based write operation over external conversational memory.

We embed entities, attributes, values, and query text using an encoder $\phi(\cdot) \in \mathbb{R}^d$, with all embeddings ℓ_2-normalized. In our implementation, ϕ is `all-mpnet-base-v2`, a Sentence-Transformers model based on MPNet representations (Reimers & Gurevych, 2019; Song et al., 2020). The memory system must support three operations: inserting new claims, revising existing claims when an update occurs, and retrieving candidate claims for a query.

3.2. From Flat Memory to Structured Revision

A flat retrieval memory stores every observed claim as an independent entry:

$$\mathcal{M}_{\text{flat}} = \{(e_t, a_t, v_t, \phi(v_t))\}_{t=1}^{T}.$$

At query time, it retrieves the top-K entries by cosine similarity between a query embedding $\phi(q)$ and stored value embeddings. This follows the general non-parametric memory view of retrieval-augmented models (Lewis et al., 2020), but it is insufficient for mutable conversational facts: if both old and new values remain in memory, semantic similarity alone does not determine which value is current.

HAM-GR builds up from this flat memory in three steps. First, it organizes claims in a non-parametric entity-attribute-claim hierarchy, replacing global flat search with structured routing. Second, it introduces an overwrite rule so that new claims can revise existing leaves rather than only append new entries. Third, it replaces raw overwrites with a learned gated revision module, allowing the memory embedding to encode update history. A conflict-aware extension further modulates forgetting when the old and new claims are semantically contradictory.

3.3. Non-Parametric Hierarchical Memory

Motivated by structured and adaptive memory systems for LLM agents (Xu et al., 2025; Jiménez Gutiérrez et al., 2025;

Tan et al., 2025), HAM-GR organizes memory as

$$\begin{aligned}\text{Root} &\to \text{Entity}\\ &\to \text{Attribute–claim cluster}\\ &\to \text{Claim cluster}\\ &\to \text{Claim leaf.}\end{aligned}$$

Each entity node stores an entity embedding $\phi(e)$. Each second-layer cluster stores a centroid in a mixed attribute-value space. For a claim $c = (e, a, v)$, we define

$$\mathbf{m}(a, v) = \frac{(1-\beta)\phi(a) + \beta\phi(v)}{\|(1-\beta)\phi(a) + \beta\phi(v)\|_2},$$

where $\beta \in [0, 1]$ controls the relative weight on value content. Lower-level claim clusters store centroids in value-embedding space. Claim leaves store the structured label (e, a, v), the raw value embedding $\phi(v)$, and a leaf identifier used by the learned revision state.

Insertion proceeds top-down. Given a new claim (e, a, v), HAM-GR first routes to the nearest entity node:

$$e^* = \arg\max_{n \in \mathcal{E}} \phi(e)^\top n_{\text{emb}}.$$

If the similarity is below an entity threshold τ_e, a new entity node is created. Otherwise, the claim is routed to e^*. Under this entity node, the claim is routed to the second-layer cluster with maximum similarity to $\mathbf{m}(a, v)$. In deeper layers, routing uses value similarity:

$$\ell^* = \arg\max_{\ell \in \text{children}(u)} \phi(v)^\top \ell_{\text{centroid}},$$

until a leaf-bearing node is reached.

To keep the tree balanced, each cluster has a fixed capacity. If a second-layer node exceeds capacity B or a lower-level node exceeds capacity C, its children are split into two sub-clusters using k-means (Lloyd, 1982). For a resulting cluster $\mathcal{C}_r$, the centroid is

$$\boldsymbol{\mu}_r = \frac{\sum_{\ell \in \mathcal{C}_r} \ell_{\text{emb}}}{\left\|\sum_{\ell \in \mathcal{C}_r} \ell_{\text{emb}}\right\|_2}.$$

After each insertion, centroids are updated upward. Second-layer centroids are recomputed in mixed attribute-value space, while lower-level centroids are recomputed in value-embedding space.

3.4. Overwrite-Based Belief Revision

The tree structure narrows the write location before any overwrite decision is made. Given a new claim (e, a, v), insertion first routes through the entity node, the second-layer attribute–claim cluster, and any lower claim clusters until it reaches a leaf-bearing node u. The overwrite rule is then applied only to the leaves under this locally routed node. Thus, overwrite is a local revision operation conditioned on the routing path, not a global search over all stored claims.

Let $\text{Leaves}(u)$ denote the leaves available at the routed node. HAM-GR first applies an attribute pre-filter:

$$\mathcal{L}_a(u) = \left\{\ell \in \text{Leaves}(u) : \phi(a)^\top \phi(a_\ell) \geq \tau_a\right\}.$$

This filter prevents locally co-located claims about different attributes from overwriting one another.

Among the filtered local candidates, HAM-GR selects the nearest value:

$$\ell^* = \arg\max_{\ell \in \mathcal{L}_a(u)} \phi(v)^\top \ell_{\text{emb}}.$$

If $\mathcal{L}_a(u)$ is nonempty and

$$\phi(v)^\top \ell^*_{\text{emb}} \geq \tau,$$

the new claim is treated as a revision of ℓ^*. In the non-parametric overwrite variant, the leaf label is updated to (e, a, v) and the stored embedding is replaced by $\phi(v)$. If $\mathcal{L}_a(u)$ is empty or the best local candidate falls below τ, a new leaf is appended under u.

This local overwrite rule provides an efficient structured revision baseline, but it is not guaranteed to find every stale claim in the tree. If a previous claim about the same entity-attribute pair was routed into a different subtree, it will not be considered as an overwrite candidate for the current insert. HAM-GR therefore relies on the hierarchy to make candidate revision local and efficient, while the overwrite threshold determines whether the locally retrieved leaf represents the same memory slot.

However, replacing the old embedding with the raw new value embedding discards the trajectory of the memory state. It cannot distinguish a minor refinement from a strong contradiction, nor can it learn how much old information should remain useful for future retrieval.

3.5. Soft-Gated Revision

HAM-GR replaces raw embedding overwrite with a learned gated update. The memory maintains a differentiable leaf-state dictionary

$$\mathcal{S} : \text{node\_id} \to \mathbb{R}^d,$$

where $\mathcal{S}[\ell]$ is the current learned embedding of leaf ℓ. The tree still uses raw embeddings and centroids for non-differentiable routing, while the attention module uses the learned leaf states for candidate ranking.

When a new claim overwrites an existing leaf ℓ^*, the revision module receives the incoming value embedding

$$\mathbf{e}_m = \phi(v)$$

and the previous leaf state

$$\mathbf{h}_u = \mathcal{S}[\ell^*].$$

The previous state is detached before the gate call, so gradients do not propagate through the entire sequence of historical writes. Let

$$\mathbf{x} = [\mathbf{e}_m; \mathbf{h}_u] \in \mathbb{R}^{2d}.$$

Using an LSTM-style gating mechanism (Hochreiter & Schmidhuber, 1997), the revision module computes

$$\mathbf{i} = \sigma(\mathbf{W}_i\mathbf{x} + \mathbf{b}_i), \qquad \mathbf{f} = \sigma(\mathbf{W}_f\mathbf{x} + \mathbf{b}_f),$$

$$\tilde{\mathbf{h}} = \tanh(\mathbf{W}_c\mathbf{x} + \mathbf{b}_c),$$

and updates the leaf state by

$$\mathbf{h}_{\text{new}} = \frac{\mathbf{f} \odot \mathbf{h}_u + \mathbf{i} \odot \tilde{\mathbf{h}}}{\left\| \mathbf{f} \odot \mathbf{h}_u + \mathbf{i} \odot \tilde{\mathbf{h}} \right\|_2}.$$

The input gate $\mathbf{i}$ controls how much new information enters the memory state, while the forget gate $\mathbf{f}$ controls how much prior state is retained. The revised state is written back as

$$\mathcal{S}[\ell^*] \leftarrow \mathbf{h}_{\text{new}}.$$

When a new leaf is appended, no revision is needed and the state is initialized as

$$\mathcal{S}[\ell] \leftarrow \phi(v).$$

This mechanism allows the memory embedding to encode revision history rather than only the latest raw claim. The tree decides where an update should occur, while the gate learns how the corresponding memory representation should change.

3.6. Conflict-Aware Gated Revision

Not all overwrites require the same amount of forgetting. A new claim may refine an earlier value, or it may directly contradict it. HAM-GR therefore augments the gate with a scalar conflict signal.

For an overwrite from prior leaf (e, a', v') to new claim (e, a, v), we define

$$\gamma = \left[\phi(a)^\top \phi(a')\right] \cdot \max\left(0, 1 - \phi(v)^\top \phi(v')\right).$$

The first term is large when the new and old claims refer to the same attribute. The second term is large when their values are semantically dissimilar. Their product is therefore high when the update is likely to be a true contradiction of the same memory slot.

The conflict-aware gate subtracts this signal from the forget-gate logit:

$$\mathbf{f} = \sigma(\mathbf{W}_f\mathbf{x} + \mathbf{b}_f - \lambda\gamma),$$

where λ is a learned scalar initialized to a positive value. A larger conflict score lowers the forget gate, encouraging the revised state to suppress the previous value. When the conflict score is small, the update behaves like the plain soft gate. All other gate equations remain unchanged.

3.7. Tree Retrieval

For a query q targeting (e_q, a_q), we compute the query text embedding $\phi(q)$ and define a mixed query vector

$$\mathbf{m}_q = \frac{(1-\beta)\phi(a_q) + \beta\phi(q)}{\|(1-\beta)\phi(a_q) + \beta\phi(q)\|_2}.$$

We use the query text embedding rather than a value embedding because the desired value is unknown at query time.

HAM-GR retrieves candidates by traversing the tree. At the entity layer, it scores entity nodes using $\phi(e_q)$. At the second layer, it scores clusters using $\mathbf{m}_q$. At lower layers and leaves, it scores using $\phi(q)$. In the default hierarchical beam search, the model keeps the top E entity nodes, the top S second-layer clusters under each selected entity, and the top H lower-level clusters at each depth. Once leaves are reached, all collected leaves are scored and the top K candidates are returned.

We also consider a branch-and-bound retrieval variant inspired by classical nearest-neighbor pruning (Fukunaga & Narendra, 1975). Each cluster node ℓ maintains a centroid $\boldsymbol{\mu}_\ell$ and radius r_ℓ. For query vector $\mathbf{z}_q$, we use the pruning score

$$\text{UB}(\ell, q) = \mathbf{z}_q^\top \boldsymbol{\mu}_\ell + \eta r_\ell,$$

where η is fixed. Nodes are expanded in decreasing order of this score. A subtree is pruned when its score falls below the current top-K leaf threshold. Since the radius-based score is used as a practical pruning heuristic in the normalized embedding space, we do not rely on it as a formal guarantee.

Compared with flat retrieval systems whose candidate pool grows linearly with the number of stored claims, HAM-GR constrains both retrieval and overwrite to localized subtrees. This reduces stale candidate proliferation and limits revision decisions to bounded local neighborhoods rather than the full memory store. Although the current work does not benchmark wall-clock retrieval latency, the hierarchical organization is intended to scale more favorably than append-only retrieval as conversational histories grow.

3.8. Attentive Candidate Ranking and Training

The tree returns a candidate set

$$\mathcal{C}_q = (\ell_1, \ldots, \ell_K).$$

For each candidate leaf, HAM-GR retrieves its learned state from the leaf-state dictionary:

$$\mathbf{h}_j = \mathcal{S}[\ell_j].$$

Let

$$\mathbf{H} = [\mathbf{h}_1, \ldots, \mathbf{h}_K]^\top \in \mathbb{R}^{K \times d}.$$

Candidate ranking uses scaled dot-product attention (Vaswani et al., 2017). A learned projection $\mathbf{W}_k \in \mathbb{R}^{d_{\text{proj}} \times d}$ maps both the query and candidate states into a shared space:

$$\mathbf{k}_j = \mathbf{W}_k \mathbf{h}_j, \qquad \mathbf{r}_q = \mathbf{W}_k \phi(q).$$

The candidate score and attention distribution are

$$s_j = \frac{\mathbf{r}_q^\top \mathbf{k}_j}{\sqrt{d_{\text{proj}}}}, \qquad \alpha_j = \frac{\exp(s_j)}{\sum_{j'=1}^{K} \exp(s_{j'})}.$$

At inference time, the model predicts the value stored at

$$\hat{j} = \arg\max_j \alpha_j.$$

For abstention, we threshold the maximum raw pre-softmax score

$$s_{\hat{j}},$$

rather than the normalized attention weight $\alpha_{\hat{j}}$. The system abstains when $s_{\hat{j}}$ falls below a threshold selected on validation data. This avoids treating a candidate as reliable merely because it has high relative probability among weak candidates.

The training loss is applied only to answerable, non-drift queries. Drift-check queries are excluded from the ranking loss and are handled at inference through the abstention threshold. For each supervised query, HAM-GR selects a local target index j^* from the retrieved candidate set using a two-stage rule. First, it searches for an exact match:

$$(e_{\ell_j}, a_{\ell_j}, v_{\ell_j}) = (e_q, a_q, \hat{v}(q)).$$

If such a candidate exists, its index is used as j^*. If no exact match is retrieved, HAM-GR falls back to slot-level supervision by selecting the first candidate that matches the target entity and attribute:

$$(e_{\ell_j}, a_{\ell_j}) = (e_q, a_q).$$

This fallback provides weaker supervision: it teaches the attention module to localize the correct entity-attribute slot even when the retrieved candidate contains a stale value. If neither an exact-match candidate nor a same-slot candidate appears in $\mathcal{C}_q$, the query is skipped for the ranking loss.

Given the selected target index j^*, the per-query loss is

$$\mathcal{L}_q = -\log(\alpha_{j^*} + \epsilon), \qquad \epsilon = 10^{-8}.$$

The episode loss is the mean over supervised queries:

$$\mathcal{L} = \frac{1}{|\mathcal{Q}_{\text{sup}}|} \sum_{q \in \mathcal{Q}_{\text{sup}}} \mathcal{L}_q.$$

Gradients flow through the attention scores into $\mathbf{W}_k$ and into candidate states $\mathbf{h}_j$. For overwritten leaves, $\mathbf{h}_j = \mathbf{h}_{\text{new}}$ retains a computational path through the gated revision module, so retrieval supervision also updates $\mathbf{W}_i$, $\mathbf{W}_f$, $\mathbf{W}_c$, and the conflict parameter λ. Tree routing, threshold decisions, centroid updates, and branch-and-bound traversal are non-differentiable and treated as fixed computation during each forward pass.

4. Experiments

4.1. Experimental Questions

We design experiments around three questions.

Q1. Does structured hierarchical memory reduce stale retrieval compared with flat vector memory, and does local overwrite further suppress stale candidates?

Q2. Does learned soft-gated revision improve final answer accuracy over the non-parametric overwrite baseline, and does gradient flow through revised leaf embeddings provide a useful training signal?

Q3. When does conflict-aware forgetting help, and how does the overwrite threshold τ control the trade-off between aggressive revision and false overwrite?

4.2. Synthetic Conversational Memory Benchmark

Standard dialogue benchmarks do not isolate belief revision as a directly measurable property. Gold labels are usually not keyed to individual entity-attribute slots, update histories are not explicitly tracked, and retrieval failures are difficult to separate from reasoning failures. We therefore construct a synthetic benchmark in which every fact's provenance, revision history, and current status are known by construction. This allows us to evaluate whether a memory system retrieves current facts, suppresses stale facts, preserves unrelated facts, distinguishes confusable entities, and abstains when a queried fact was never stated. Although synthetic, the benchmark isolates belief revision under controlled update histories, allowing stale retrieval, contradiction behavior, and overwrite dynamics to be measured directly. This controlled setting makes it possible to separate memory revision failures from unrelated language-generation or reasoning errors.

Each episode simulates a multi-turn conversation about a group of people. A claim is a structured triple (e, a, v), where e is a person, a is one of 12 attributes, and v is drawn from an attribute-specific value pool. The attributes are city,

Table 1. Synthetic benchmark length buckets.

Bucket	Turns	Entities	Updates	Distractors
xs	≤ 15	2–3	1–2	1–3
s	16–35	4–6	2–4	4–8
m	36–65	7–10	5–8	10–16
l	66–100	11–14	9–14	18–26
xl	> 100	15–20	15–22	28–40

job, school, favorite food, hobby, employer, pet, sport, dietary restriction, neighborhood, relationship status, and language. An episode begins with an initial sequence of claim insertions, followed by interleaved updates, distractor turns, and chain updates in which the same entity-attribute slot is revised multiple times. The gold memory state records, for every observed entity-attribute pair, the most recent asserted value.

The benchmark contains 7,000 episodes: 5,000 for training, 1,000 for validation, and 1,000 for test. Episodes average 108.5 turns in the training split and 109.7 turns in the validation split. We generate five length buckets to control episode complexity, with longer episodes containing more entities, more updates, and more distractor turns.

Length buckets are mixed in each split with proportions 10% xs, 15% s, 20% m, 25% l, and 30% xl. This distribution intentionally weights the benchmark toward longer conversations. The difficulty distribution is 30% easy, 40% medium, and 30% hard. Easy episodes contain no confusable entity names and no repeated updates to the same fact. Medium episodes introduce one confusable entity pair, such as "Alice" and "Alicia," and use a 20% re-update probability. Hard episodes contain one or two confusable entity pairs and use a 35% re-update probability, creating chains in which a fact may be revised multiple times.

Each episode generates up to four query types.

Update check. The query asks for the current value of an entity-attribute pair that has been revised at least once. This tests whether the system tracks belief revision and suppresses stale values.

Preservation check. The query asks for a fact that was never updated in the episode. This tests whether revision operations contaminate unrelated memory slots.

Entity confusion check. The query asks about one member of a confusable name pair when the two entities have different values for the same attribute. This tests whether the memory routes and retrieves using the intended entity rather than a similar name. This query is generated only when the episode contains a suitable confusable pair.

Drift check. The query asks about an entity-attribute pair that was never mentioned. The correct response is abstention, denoted by $\varnothing$. This tests whether the system avoids producing unsupported answers.

Update, preservation, and drift checks are generated for every episode. Entity confusion checks are generated only when a suitable confusable pair with divergent attribute values exists.

4.3. Baselines and Model Variants

We compare models that incrementally add the components introduced in Section 3. This construction path isolates the effect of structured routing, local overwrite, learned gated revision, and conflict-aware forgetting.

Flat Retrieval. This baseline stores every observed claim as an independent memory entry and retrieves the top-K entries using cosine similarity between the query embedding $\phi(q)$ and stored value embeddings. It uses no tree structure, no overwrite, and no learned ranking. This corresponds to a standard dense non-parametric memory baseline (Lewis et al., 2020).

Tree + Append. This variant routes insertions through the hierarchical tree but appends every new claim as a new leaf. No existing leaf is overwritten. At retrieval time, leaves are ranked by cosine similarity without a learned attention module. This variant isolates whether tree-based organization helps when memory remains append-only.

Tree + Overwrite. This variant adds the local overwrite rule with threshold $\tau = 0.65$. A new claim overwrites the nearest attribute-compatible leaf under the locally routed node if their value similarity exceeds τ. This model has no learned components and serves as the non-parametric revision baseline.

Gated-Hier and Gated-BnB. These variants add the soft-gated revision module and learned attention ranker on top of Tree + Overwrite. The gate produces revised leaf states $\mathbf{h}_{\text{new}}$ at insertion time, and the attention module ranks retrieved candidates at query time. We evaluate two retrieval strategies: hierarchical beam search and branch-and-bound search. These variants do not use the conflict signal.

HAM-GR-Hier and HAM-GR-BnB. These are the full HAM-GR variants. They extend the gated models with conflict-aware forgetting by adding the learned scalar λ and the conflict signal

$$\gamma = [\phi(a)^\top \phi(a')] \cdot \max(0, 1 - \phi(v)^\top \phi(v')).$$

We evaluate both hierarchical beam search and branch-and-bound retrieval.

4.4. Evaluation Metrics

Accuracy. Accuracy measures whether the final predicted answer exactly matches the gold answer. For drift checks,

the prediction is correct if and only if the system abstains.

Stale Retrieval Rate. Stale retrieval rate, or SRR, measures contamination in the retrieved candidate set. For an answerable query targeting (e_q, a_q), SRR is the fraction of top-K candidates whose entity-attribute label matches (e_q, a_q) but whose value is not the current gold value. A high SRR indicates that outdated values remain present in the retrieval pool.

Contradiction Rate. Contradiction rate, or CR, measures the fraction of retrieved candidates that directly conflict with the gold answer for answerable queries. A candidate is contradictory if it matches the queried entity-attribute slot but contains a different value from the gold answer.

Accuracy, SRR, and CR capture different aspects of memory quality. A model may answer correctly while still retrieving stale candidates at lower ranks. Conversely, a model may have a clean candidate set but fail to rank the correct candidate first. Reporting all three metrics separates final answer correctness from the quality of the memory retrieval pool.

4.5. Implementation Details

All embeddings are produced by `all-mpnet-base-v2` with dimension $d = 768$ (Reimers & Gurevych, 2019; Song et al., 2020). The attention projection uses $d_{\text{proj}} = 768$, and the retrieval candidate set size is $K = 20$.

Tree thresholds are $\tau_e = 0.92$, $\tau_a = 0.80$, and $\tau = 0.65$ unless otherwise stated. Cluster capacities are $B = 3$ for second-layer nodes and $C = 6$ for lower-level nodes. When a cluster exceeds capacity, it is split with k-means using $k = 2$ and 5 restarts (Lloyd, 1982). Hierarchical beam search keeps the `TOP_E` = 2 entity nodes, `TOP_S` = 3 second-layer clusters per entity, and `TOP_H` = 3 lower-level clusters per depth. Although restricting retrieval to a single entity node (`TOP_E` = 1) maximizes routing precision, we set `TOP_E` = 2 for all main evaluations to simulate the ambiguity introduced by imperfect entity extraction in realistic LLM pipelines, where the correct entity node is not always ranked first. Top Branch-and-bound uses radius coefficient $\lambda_{\text{radius}} = 1.0$ and an expansion budget of 500.

The trainable parameters are the gate projections $\mathbf{W}_i$, $\mathbf{W}_f$, $\mathbf{W}_c$, the attention projection $\mathbf{W}_k$, and, for HAM-GR, the conflict scalar λ. We optimize these parameters with AdamW (Loshchilov & Hutter, 2019) using learning rate 3×10^{-4} and weight decay 10^{-4}. The learning rate follows cosine annealing over 30 epochs (Loshchilov & Hutter, 2017). Gradients are accumulated over 16 episodes per optimizer step. Training uses early stopping on held-out validation loss with patience 5. The abstention threshold is selected post-hoc on the validation set by grid search over $\{0.20, 0.25, \ldots, 0.75\}$. All experiments are run on a single NVIDIA A100 GPU. All trainable parameters consist of the gated revision module (W_i, W_f, W_c; 3,542,785 parameters) and the learned attention projection W_k (1,179,648 parameters), totaling 4,722,433 parameters; conflict-aware variants add a single scalar λ, for 4,722,434 parameters in total.

Table 2. Main validation results.

Model	Accuracy	SRR	CR
Flat Retrieval	0.507	0.387	0.186
Tree + Append	0.509	0.395	0.187
Tree + Overwrite	0.631	0.087	0.035
Gated-Hier (best τ)	0.792	0.0079	0.0023
Gated-BnB (best τ)	0.788	0.0089	0.0019
HAM-GR-Hier (best τ)	0.807	0.0079	0.0023
HAM-GR-BnB (best τ)	0.789	0.0089	0.0019

4.6. Main Results

Table 2 reports validation accuracy, stale retrieval rate, and contradiction rate for the main model variants. The results show a clear progression: hierarchical routing alone is insufficient, local overwrite removes most stale contamination, and learned gated revision provides the largest improvement in final answer accuracy.

Tree routing alone does not solve revision. Tree + Append performs nearly identically to Flat Retrieval, with accuracy changing only from 0.507 to 0.509. Its stale retrieval rate is also similar, increasing slightly from 0.387 to 0.395. This shows that hierarchical organization by itself does not remove stale information when every claim is retained indefinitely. More broadly, these results suggest that belief revision is fundamentally a write-time memory management problem rather than solely a retrieval-ranking problem. Hierarchical structure alone provides little benefit when stale and current claims continue to coexist in the candidate pool; the largest gains emerge only once overwrite actively suppresses outdated same-slot claims. Old and new values can still coexist in the memory, and retrieval over an append-only candidate pool cannot reliably distinguish the current value from superseded ones.

Local overwrite is the decisive structural step. Adding local overwrite increases accuracy from 0.507 to 0.631, a 12.4-point absolute gain over Tree + Append. At the same time, SRR drops from 0.387 to 0.087, and CR drops from 0.186 to 0.035. This confirms that stale contamination is the dominant failure mode of flat and append-only memory. When the memory replaces locally matched stale leaves rather than accumulating all historical claims, the retrieved candidate set becomes substantially cleaner.

Gated revision improves beyond non-parametric overwrite. The Gated and HAM-GR variants reach around 0.790 validation accuracy at their best overwrite threshold,

Table 3. Ablation path from flat retrieval to HAM-GR.

Model	Tree	Overwrite	Gate	Conflict
Flat Retrieval				
Tree + Append	✓			
Tree + Overwrite	✓	✓		
Gated	✓	✓	✓	
HAM-GR	✓	✓	✓	✓

improving over Tree + Overwrite by 15.9 points and over Flat Retrieval by 28.3 points. Since Tree + Overwrite already removes most stale candidates, this additional gain cannot be explained only by candidate-pool cleaning. Instead, it suggests that the revised embeddings $\mathbf{h}_{\text{new}}$ encode useful update history that raw value embeddings do not capture, and that the learned projection $\mathbf{W}_k$ can exploit this geometry during candidate ranking.

Conflict-aware gating and retrieval strategy. At the best selected threshold, the conflict-aware HAM-GR variants and the plain Gated variants obtain similar overall validation accuracies of about 0.790. This suggests that, under the cleanest operating regime, the soft gate already captures much of the useful revision signal. We analyze this behavior through threshold sweeps and update-heavy episodes below.

4.7. Ablation: From Flat Retrieval to HAM-GR

To isolate the contribution of each component, Table3 summarizes the construction path from flat retrieval to the full HAM-GR model. Each step adds one mechanism: structured routing, local overwrite, learned gated revision, and conflict-aware forgetting.

The flat-to-append comparison shows that routing claims through a hierarchy is not sufficient when the memory remains append-only. The append-to-overwrite transition is the largest structural change: it removes stale same-slot leaves from the candidate pool and reduces SRR by 77.9% relative to Tree + Append. The overwrite-to-gate transition adds a learned representation layer on top of this cleaner memory structure. The gate does not replace overwrite; rather, it depends on overwrite to identify where revision should occur, and then learns how the corresponding leaf representation should change.

This separation is important. The tree controls which memory states survive as local candidates, while the gate controls what those surviving states encode. The non-parametric overwrite rule provides discrete revision behavior, and the learned gate turns that revision event into a differentiable embedding update. HAM-GR therefore combines symbolic-like slot localization with neural revision of the leaf representation.

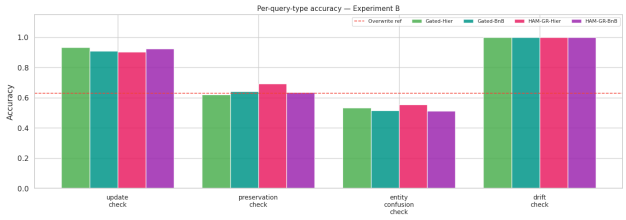

Figure 1. Validation accuracy broken down by query type. HAM-GR and Gated models show the largest gains over baselines on update checks, where stale contamination in append-only memory is the dominant failure mode.

4.8. Analysis by Query Type and Episode Length

Query type breakdown. The four query types probe different memory failures. Update checks directly measure belief revision: the system must return the latest value after one or more changes. Preservation checks test whether revision operations disturb unrelated facts. Entity confusion checks test whether similar entity names pull the model toward the wrong slot. Drift checks test abstention calibration when the target entity-attribute pair was never observed.

The abstention threshold was selected by grid search over $\{0.20, 0.25, ..., 0.75\}$ on the validation set. Accuracy peaks in the range of abstain threshold 0.20–0.40 for all gated variants, where the model answers confidently on retrievable queries while abstaining on drift checks, and degrades steadily as the threshold rises and the model becomes overly conservative, abstaining on answerable queries it should resolve. Notably, SRR and CR remain flat across the entire threshold sweep for all gated models, confirming that abstention calibration is independent of candidate pool quality. The threshold controls only the final answer decision, not which candidates are retrieved. All main results report accuracy at the threshold that maximizes validation accuracy, which falls at 0.4 for HAM-GR-Hier.

Flat Retrieval fails most severely on update and entity confusion checks. For update checks, old and new values remain in the same retrieval pool, so the model often retrieves stale same-slot candidates. For entity confusion checks, semantically similar names can pull claims from the wrong entity into the top-K set. Tree + Overwrite improves update behavior by removing many stale same-slot leaves, and it partially improves confusion behavior by routing claims through entity nodes before lower-level retrieval. HAM-GR further improves ranking because revised embeddings encode the history of local updates, making current leaves easier to separate from stale or nearby alternatives.

Episode length. Episode length increases the difficulty of conversational memory because more turns introduce more entities, more updates, and more distractors. Flat Retrieval degrades most sharply in the longest episodes,

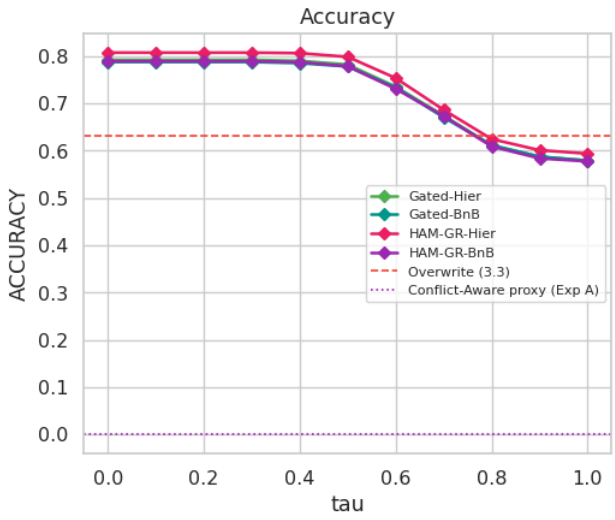

Figure 2. Validation accuracy across overwrite thresholds τ with learned parameters fixed. Aggressive overwrite ($\tau \leq 0.50$) sustains $\tilde{0}.79$ accuracy; conservative overwrite ($\tau \to 1.00$) degrades to 0.58 as the tree approaches append-only behavior.

where the candidate pool contains many more stale and semantically related claims. Tree + Overwrite is more robust because local revision prevents the number of leaves for a frequently updated slot from growing linearly with the number of updates. HAM-GR preserves this advantage while adding learned revision, with the largest gains in long episodes where repeated updates create meaningful revision trajectories in $\mathbf{h}_{\text{new}}$.

4.9. Effect of Overwrite Threshold

The overwrite threshold τ controls how aggressively the tree revises existing leaves. Lower values make overwrite more likely, producing a cleaner but potentially over-aggressive memory. Higher values make overwrite more conservative, reducing false overwrites but allowing stale duplicates to remain in the tree.

We train the gated models with default threshold $\tau = 0.65$ and evaluate them over a sweep

$$\tau \in \{0.00, 0.10, \ldots, 1.00\},$$

holding learned parameters fixed. This isolates the effect of the non-parametric overwrite policy from the learned gate and attention weights. The sweep found in Figure 2 reveals three regimes. For $\tau \leq 0.50$, accuracy is approximately 0.79. In this aggressive-overwrite regime, most inserts revise existing leaves, the tree remains clean, and gated revision improves retrieval even when stale coexistence is minimal. Around the training threshold $\tau = 0.65$, accuracy declines to approximately 0.70. In this intermediate regime, some updates become appends, stale and current leaves coexist, and the attention module must filter a noisier candidate set. For $\tau > 0.65$, accuracy declines further, reaching approximately 0.58 at $\tau = 1.00$. In this conservative-overwrite regime, the tree approaches append-only behavior and stale contamination becomes the dominant source of error.

The exceptional performance observed in the aggressive-overwrite regime ($\tau \leq 0.50$), where accuracy reaches approximately 0.79, suggests a substantial advantage of leveraging learned gated representations over purely structural accumulation. By lowering the threshold, the system forces almost all incoming same-slot updates to revise existing leaves rather than appending as new entries. This aggressive policy ensures the tree remains exceptionally clean by preventing the proliferation of stale, superseded facts. Consequently, instead of forcing the attention module to untangle multiple coexisting claims, the memory consolidates the revision history into a single, dynamically updated leaf. The sustained high accuracy in this regime suggests that the soft-gated revision module is highly capable of blending and refining these continuous updates into a robust embedding. The importance of gated revision is further supported by the degradation observed when overwrite becomes increasingly conservative and the gate is invoked less frequently. As the threshold increases past 0.65 and approaches 1.00, accuracy sharply declines to approximately 0.58. In this conservative regime, the model defaults to append-only behavior, effectively starving the learned gate of opportunities to act and allowing stale contamination to become the dominant source of error. This steep degradation suggests that structural accumulation alone is insufficient; rather, it underscores the core design philosophy of HAM-GR: the strongest retrieval performance occurs when structural mechanisms aggressively eliminate stale accumulation, allowing the learned gate to fully dictate the nuanced representation of evolving memory states.

Effect of Conflict-Aware Scalar While the conflict-aware scalar (λ) is introduced as a targeted mechanism for handling contradictory updates, the empirical evidence suggests it does not provide a meaningful performance advantage. As demonstrated in the threshold sweep in Figure 2, the accuracy curves for the conflict-aware HAM-GR variants and the plain Gated variants are virtually indistinguishable across all values of τ . Even at the optimal operating threshold, both models achieve a similar overall validation accuracy of approximately 0.79. This consistent parity across the entire spectrum, from aggressive overwriting regimes to conservative, append-heavy regimes, indicates that λ is not empirically necessary for the model's success. Instead, the data implies that the standard soft-gated revision module is already sufficiently capable of learning to suppress stale or contradictory information independently, rendering the explicit mathematical penalty of λ functionally redundant in terms of overall accuracy.

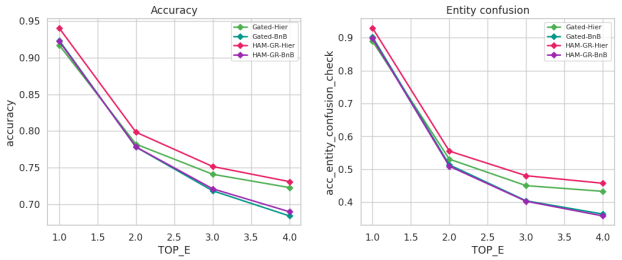

Figure 3. Validation accuracy and entity confusion accuracy across TOP_E values. Restricting to TOP_E = 1 yields approximately 94% accuracy by eliminating cross-entity contamination; main results use TOP_E = 2 to simulate realistic extraction noise.

To determine whether the conflict-aware gate provides any benefit in harder revision settings, we compared HAM-GR and Gated variants specifically on hard episodes, which contain one or two confusable entity pairs and a 35% re-update probability. Even in this subset, the two models achieve comparable accuracy (77.5% vs. 77.6%), suggesting that the standard soft gate learns sufficient forgetting behavior from retrieval supervision alone, even when updates are repeated and contradictions are more frequent. We therefore treat the conflict scalar as redundant under the current benchmark conditions, while noting that it may provide benefit in naturalistic settings where supervision is noisier and updates are more indirect.

Overall, the threshold sweep supports the central design of HAM-GR. Local overwrite controls whether stale leaves remain in memory, while gated revision controls how overwritten leaves are represented. The strongest performance is obtained when the tree is aggressive enough to avoid stale accumulation and the learned gate is used to shape the representation of revised memory states.

4.10. Effect of Entity Expansion Bounds

The parameter sweep over `TOP_E` (Figure 3) reveals a dramatic performance shift when the tree routing is strictly bottlenecked. When the search is restricted to a single top-level entity node (`TOP_E` = 1), overall accuracy across all models surges to a peak of approximately 94%. This improvement is almost entirely driven by a massive increase in entity confusion accuracy, which jumps to roughly 90% at `TOP_E` = 1 compared to the 50% baseline observed at `TOP_E` = 2, the setting used for full evaluations. Interestingly, this heavily restricted search does coincide with a higher Stale Retrieval Rate (SRR), rising to 0.023 at `TOP_E` = 1 from 0.020 at `TOP_E` = 2. A likely explanation for this slight SRR increase is that by forcing all retrieval through a single entity path, the model occasionally traps itself in a subtree containing a stale update if the true update was misrouted during insertion. However, this marginal increase in stale retrieval is vastly outweighed by the 10+ point gain in overall accuracy, demonstrating that preventing entity contamination is the most critical factor for success.

Geometric Blending and the Role of Extraction Noise The sharp degradation in performance when expanding the entity search space (`TOP_E` $\geq$1) suggests a critical vulnerability of soft-gated revision: geometric blending. While the soft gate effectively encodes revision history, this continuous blending of states may cause the resulting leaf embeddings to become less geometrically distinct than those produced by a stark, non-parametric overwrite. Consequently, when the retrieval pool is expanded to include confusable entities, these subtly blended representations become difficult for the attention module to differentiate, leading to a spike in entity confusion. This suggests that the hierarchical tree structure plays an important role in isolating the learned gate from cross-entity contamination; the rigid routing at the top layer acts as a vital shield, isolating the learned gate from cross-entity contamination. In our experiments, we deliberately evaluated models at `TOP_E = 2` to simulate the inevitable noise introduced by real-world LLM entity extraction, as our synthetic benchmark relies on clean entity labels. However, this finding suggests a highly promising scaling trajectory: if future implementations employ robust LLM pipelines capable of highly precise entity extraction, effectively allowing the system to operate safely at `TOP_E = 1`, the advantages of the HAM-GR architecture could be dramatically amplified. Further research is required to evaluate alternative embedding generation strategies and to determine how robustly the tree structure scales when exposed to organic, non-synthetic extraction noise.

5. Conclusion

Long-form conversational agents face a memory problem that goes beyond retrieval: when facts change mid-conversation, systems must not only find relevant information but also determine which version of that information is current. Flat retrieval-augmented memory fails at this task because semantic similarity cannot distinguish an updated value from the stale value it replaced. We introduced HAM-GR to address this specific failure mode through a three-component architecture: hierarchical routing, local overwrite, and learned gated revision. Each targets a distinct source of memory error.

Our ablation results establish that these components address different aspects of the problem. The transition from flat retrieval to hierarchical routing with local overwrite accounts for the largest reduction in stale contamination, dropping the stale retrieval rate from 38.7% to 8.7% and raising accuracy from 50.7% to 63.1%. This finding has a clear interpretation: the dominant failure mode of append-only memory is not a ranking problem but a candidate-pool problem. When stale and current claims coexist as equally retrievable en-

tries, no downstream ranker can reliably recover. Structured overwrite addresses this at the source by preventing the accumulation of superseded leaves in the first place.

Taken together, these results suggest that the dominant challenge in long-term conversational memory is not simply retrieving relevant information, but maintaining a retrieval pool whose contents remain temporally coherent as facts evolve. In this setting, overwrite behavior contributes more to performance than hierarchical organization alone.

Learned gated revision provides a complementary and substantial gain on top of the cleaned candidate pool, improving accuracy by an additional 15.9 points over the non-parametric overwrite baseline. Because Tree + Overwrite already eliminates most stale candidates, this gain cannot be attributed to further pool cleaning. Instead, it reflects the value of encoding revision history in the leaf embedding itself: a memory state that has been updated carries a different geometric signature than one that was initialized and never revised, and the learned attention projection can exploit this structure during candidate ranking. This interpretation is supported by the threshold sweep in Figure 2, which shows that the gated models sustain high accuracy even in the aggressive-overwrite regime where nearly every update triggers a leaf revision. If the gain were due to filtering rather than representation, accuracy should degrade in this regime; it does not.

The conflict-aware extension did not provide a measurable accuracy advantage over the plain gated model at any overwrite threshold. We interpret this null result cautiously rather than as a refutation. On our synthetic benchmark, the soft gate appears sufficient to learn appropriate forgetting behavior from retrieval supervision alone. It remains an open question whether explicit conflict modulation confers benefits in naturalistic settings, where updates may be indirect, entity boundaries may be fuzzy, and supervision is noisier.

Two additional findings carry practical implications. First, the entity expansion sweep shows that accuracy peaks at approximately 94% when routing is restricted to a single entity node (`TOP_E` = 1), with entity confusion accuracy rising from roughly 50% to 90%. This suggests that the current 81% ceiling under `TOP_E` = 2 is largely attributable to entity-level cross-contamination introduced by noisy extraction, a condition we deliberately simulate to reflect realistic LLM pipelines. Systems that can reliably identify entities at insertion time should achieve substantially stronger performance. Second, the overwrite threshold analysis reveals that gated revision is most valuable precisely when overwrite is most aggressive: the revised leaf embedding encodes the sequence of updates into a single representation, eliminating the need for the ranker to adjudicate among competing stale entries.

Several limitations bound the generalizability of these results. The benchmark is fully synthetic, with clean claim structure, controlled entity names, and a closed attribute vocabulary. Real conversations involve implicit updates, paraphrastic variation, partial claims, and entities that shift in salience without being explicitly revised. The encoder used throughout, `all-mpnet-base-v2`, is a fixed sentence embedding model; HAM-GR does not update the encoder during training, and its performance is therefore bounded by the quality of these frozen representations for entity and attribute matching. Future work should evaluate HAM-GR on naturalistic dialogue datasets, investigate whether the gated revision mechanism scales to larger and more heterogeneous memory stores, and examine whether the conflict-aware gate contributes more meaningfully when supervision is sparse or when updates are indirect. Integrating HAM-GR with a full LLM pipeline, where entity extraction, claim parsing, and answer generation are all subject to error, will also be necessary to assess end-to-end utility.

Our results support a modular view of conversational memory: structural mechanisms should control which memory states remain in the retrieval pool, while learned mechanisms should control what those states encode. HAM-GR demonstrates that combining these two functions, through local overwrite and gated revision, produces substantially better belief tracking than either function alone.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Alchourrón, C. E., Gärdenfors, P., and Makinson, D. On the logic of theory change: Partial meet contraction and revision functions. *The Journal of Symbolic Logic*, 50(2): 510–530, 1985.

Fukunaga, K. and Narendra, P. M. A branch and bound algorithm for computing k-nearest neighbors. *IEEE Transactions on Computers*, C-24(7):750–753, 1975. doi: 10.1109/T-C.1975.224297.

Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

Hu, Y., Wang, Y., and McAuley, J. Evaluating memory in LLM agents via incremental multi-turn interactions. In *International Conference on Learning Representations*, 2026.

Jiménez Gutiérrez, B., Shu, Y., Qi, W., Zhou, S., and Su, Y. From RAG to memory: Non-parametric continual learning for large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

Lee, K.-H., Chen, X., Furuta, H., Canny, J., and Fischer, I. A human-inspired reading agent with gist memory of very long contexts. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 26396–26415. PMLR, 2024.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pp. 9459–9474, 2020.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

Lloyd, S. P. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. doi: 10.1109/TIT.1982.1056489.

Loshchilov, I. and Hutter, F. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 2023. doi: 10.1145/3586183.3606763.

Reimers, N. and Gurevych, I. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 3982–3992, 2019. doi: 10.18653/v1/D19-1410.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Song, K., Tan, X., Qin, T., Lu, J., and Liu, T.-Y. MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems*, volume 33, pp. 16857–16867, 2020.

Tan, Z., Yan, J., Hsu, I.-H., Han, R., Wang, Z., Le, L. T., Song, Y., Chen, Y., Palangi, H., Lee, G., Iyer, A., Chen, T., Liu, H., Lee, C.-Y., and Pfister, T. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., and Yu, D. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations*, 2025.

Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

Zhong, W., Guo, L., Gao, Q., Ye, H., and Wang, Y. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024. doi: 10.1609/aaai.v38i17.29946.